

A very beginner-friendly course

React t for Beginners

From zero to your first React app, one
small step at a time.

 Components  Hooks  APIs  Routing



Press space: countdown, explosion, then the title. Welcome everyone!

6 chapters · 1 mini project

Your learning journey



1. Meet React

What, why, when



2. Building blocks

JSX, components, props



3. How it works

Virtual DOM, React flow



4. Hooks

useState, useEffect, fetch



5. Forms

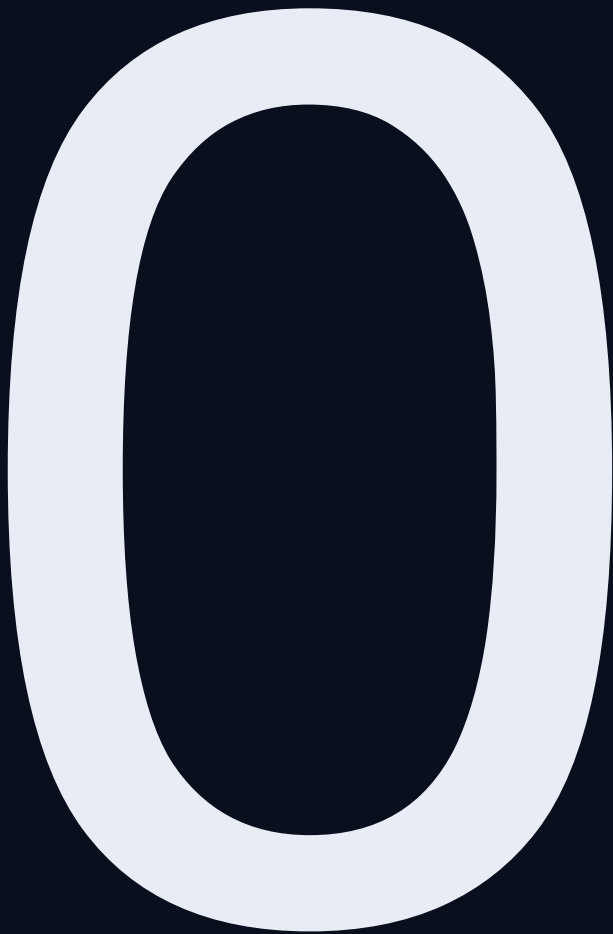
Controlled inputs



6. Routing & more

Pages, lists, conditions

Six chapters, each builds on the last. Finish line: quiz, cheat sheet and a To-Do mini project.



1

Chapter 1

Meet React

What it is, why people love it, and
when to use it.

Chapter 1: no code yet, just ideas.

The problem React solves



Without React

You change some data. Then YOU must find every place on the page that shows it and update each one by hand. Easy to forget one!



With React

You change the data. React updates the page for you, automatically. You only describe what the screen should look like.



Think of an Excel sheet: change one number and the TOTAL updates by itself.

React does the same for web pages.

In jQuery we update the DOM by hand. React flips it: update the data, React updates the screen.

Two ways of thinking: jQuery vs React

Imperative: tell HOW, step by step

script.js

// jQuery: you do every step

let count = 0;

\$("#btn").click(function () {

count = count + 1;

\$("#num").text(count); *// update by hand*

});

Declarative: describe WHAT you want

Counter.jsx

// React: describe the screen

const [count, setCount] = useState(0);

<button onClick={() => setCount(count + 1)}>

{count}

</button> *// React updates it*



Like ordering food: jQuery = cooking it yourself step by step. React = telling the waiter what you want.

You already know jQuery. Imperative means step-by-step instructions. Declarative means describing the result.

So, what is React?

React is a JavaScript
library for building
user interfaces using

components.



Library

A toolbox of ready-made code you add to your project.



User interface

Everything people see and click: buttons, forms, lists.



Components

Small reusable pieces that join to make a full page.

Three key words: library, user interface, components. React only handles the view: what the user sees.

The React story in 5 moments

2011 Created at Facebook by Jordan Walke

2013 Released free and open source

2015 React Native for mobile apps

2019 Hooks arrive (React 16.8)

2024 React 19 released

React is not new: it has been improved for over 10 years, which is why it is so stable and popular.

Why do people choose React?



Reusable pieces

Build a button or card once, use it everywhere.



Fast updates

The Virtual DOM changes only what really changed.



Easy to start

If you know HTML and JavaScript, you can learn React.



Huge community

Used by apps like Instagram, Netflix and Airbnb. Lots of jobs and help online.

React vs React.js vs React Native

React = React.js = ReactJS.
Same thing, just nicknames!

Name	What it is	Runs on
 React	The library for building user	Web browser

Name	What it is	Runs on
	interfaces	s
 React.js / ReactJS	Nickname for React (".js" = JavaScript)	Web browser s
 React Native	A DIFFERENT tool using React ideas for phone apps	Android and iOS

Students often think React and React.js are different. They are the same. React Native is the separate one.

Library vs

Framework



Library (React)

YOU are in control. You pick React for the UI, then add other tools (router, forms) as you need them.



Framework (Angular)

The framework is in control. Everything comes included, and you follow its rules.



Library = buying separate LEGO sets you choose. Framework = a full ready-made LEGO city kit with a fixed plan.

When should you choose React?

✓ Choose React when...



The page changes a lot when users click or type



The same UI pieces repeat (cards, lists)



You want an app feel with no page reloads



You want skills companies hire for

🤔 Maybe skip React when...



It's a simple page with little interaction



It's a tiny script or one small form



It's a content site needing top Google
ranking (use Next.js, built on React)

0

2

Chapter 2

The building blocks

Setup, JSX, components, props
and single page apps.

Chapter 2: the pieces every React app is made of.

Create your first React app

Step 0: install Node.js. Then type in the terminal:

Terminal

// 1. Create the project

```
npm create vite@latest my-app -- --template react
```

// 2. Go inside the folder

```
cd my-app
```

// 3. Install the packages

```
npm install
```

// 4. Start the app 🚀

```
npm run dev
```

Open localhost:5173 in your browser. 🎉



Important files

`index.html` the one and only HTML page

`src/main.jsx` starts React

`src/App.jsx` your main component

`package.json` list of packages

Vite is the fast, modern way to start React. Every save instantly updates the browser.

How a React app starts

index.html has an empty `<div id="root">`

main.jsx React takes over #root

App.jsx the main component

Your components Navbar, Card, Footer...

main.jsx

```
import { createRoot } from "react-dom/client";  
import App from "./App.jsx";  
// find <div id="root"> and draw <App /> inside it  
createRoot(document.getElementById("root")).render(  
  <App />  
);
```



The HTML page stays almost empty.

React fills it using JavaScript.

index.html has just one empty div. main.jsx tells React to render App inside it. Everything else is components.

JSX: HTML inside JavaScript

Browsers don't understand JSX. A tool called Babel turns it into normal JavaScript first.

Watch: JSX → Babel → React.createElement → the browser shows it.

5 simple rules of JSX

Rule	 Wrong	 Right
1. Return ONE parent	<code><h1/></code> <code><p/></code>	<code><div>...</div></code> or <code><>...</></code>
2. className, not class	<code>class="box"</code>	<code>className="box"</code>
3. Close every tag	<code></code>	<code></code>
4. JavaScript goes in { }	Hello name	Hello {name}

Rule	 Wrong	 Right
5. Events use camelCase	onclick	onClick

`<> </>` is a Fragment, an invisible wrapper. `class` is a reserved word in JavaScript, so JSX uses `className`.

The magic curly braces { }

Inside JSX, { } means: "run this JavaScript and show the result".

Profile.jsx

```
const name = "Ravi";
```

```
const age = 20;
```

```
<h1>Hello, {name}!</h1>
```

```
<p>Next year: {age + 1}</p>
```

```
<p>{age >= 18 ? "Adult" : "Minor"}</p>
```

```
<p>{name.toUpperCase()}</p>
```

Result on screen

Hello, Ravi!

Next year: 21

Adult

RAVI



{ } can hold values, maths, conditions and function calls. Not if statements or loops.

Components = LEGO blocks

Navbar

Sidebar

Card

Card

Card

Footer

App

Navbar

Main

Footer

Sidebar

Card ×3

Break the page into pieces. Each piece is a component. Join them to build the app!

Left: what users see. Right: the component tree React uses. App is the parent of everything.

Your first component

App.jsx

// A component is just a function

```
function Welcome() {  
  return <h1>Hello, students!</h1>;  
}
```

// Use it like an HTML tag

```
function App() {  
  return (  
    <div>  
      <Welcome />  
      <Welcome />  
    </div>  
  );  
}
```



Name starts with a CAPITAL

Welcome  welcome 



It returns JSX

JSX = what appears on screen.

Result on screen

Hello, students!

Hello, students!

Props: sending data

into a component

App.jsx

```
// props = the data you send in
function Welcome(props) {
  return <h1>Hello, {props.name}!</h1>;
}

function App() {
  return (
    <div>
      <Welcome name="Ravi" />
      <Welcome name="Priya" />
    </div>
  )
}
```

```
);  
}
```



Like function arguments

Same component, different data each time.



Props are read-only

A component can USE props, but never change them.

Result on screen

Hello, Ravi!

Hello, Priya!

Props vs State: the big difference

	 Props	 State
Come s from	The parent component	Inside the component itself
Can it chang e?	No, read-only	Yes, with its setter (setCount)

	 Props	 State
Used for	Passing data down	Remembering things that change
Example	<code><Welcome name="Ravi" ></code>	<code>const [count, setCount] = useState(0)</code>
Real life	Your name given by parents	Your mood, which changes all day

Very common exam question. Props come from outside and are read-only. State lives inside and can change.

SPA: Single Page Application

React apps load ONE page, then swap parts of it. Watch the difference:

0

3

Chapter 3

How React works inside

The DOM, the Virtual DOM and the
React flow.

Chapter 3: behind the scenes.

First: what is the DOM?

The DOM (Document Object Model) is the browser's tree of everything on the page. JavaScript changes the page by changing this tree.

html

body

h1

p

button



Updating the real DOM again and again is SLOW, like repainting a whole wall to fix one small spot.

The Virtual DOM: React's smart copy

A light copy of the page kept in memory.

React plays spot the difference, then fixes only the differences.

Virtual DOM in 4 steps

1 State changes count goes $0 \rightarrow 1$

2 New Virtual DOMA new copy in memory

3 Diffing Compare old vs new

4 Reconciliation Update only what changed



Exam words: Diffing = comparing the two virtual trees. Reconciliation = updating the real DOM to match.

Real DOM vs Virtual DOM

	 Real DOM	 Virtual DOM
What is it?	The actual page in the browser	A copy of the page in memory
Speed to change	Slow	Very fast
Update	Can redraw	Only the

	 Real DOM	 Virtual DOM
s	big parts	changed parts
Who handles it?	The browser	React
Real life	Rebuilding the whole house	Checking the blueprint first

The React flow:

one-way data loop

1. State the data your app remembers
2. Render React draws the UI
3. Event user clicks or types
4. setState your code updates the data

Round and round.

Data flows ONE way 

The heart of React. You never touch the screen directly: you change state, React re-renders.

What makes a component re-render?

"Re-render" = React runs your component function again to draw the new screen.



Its state changes

You called `setCount`, `setName...`



Its props change

The parent sent new data.



Its parent re-renders

Children redraw with the parent.



Changing a normal variable (let $x = 5$) does NOT re-render. Only state changes do!

Props go DOWN,
events go UP

Appholds the state: count

Displayshows the count

Button+1

props: count

props: onAdd()

event: clicked! calls onAdd()

Solid arrows: props go down. Dashed arrow: the child calls a function from the parent, which updates the parent's state.

Component life

cycle: born, grows, leaves



MountComponent appears on

screen for the first time



UpdateState or props change, it
re-renders



UnmountComponent is removed
from the screen

Timer.jsx

```
useEffect(() => {  
  console.log("Mounted! 🌱");  
  return () => {  
    console.log("Unmounted! 🍂"); // cleanup  
  };  
}, []);
```



The function you return from `useEffect` is the cleanup. It runs when the component leaves. Use it to stop timers.

0

4

Chapter 4

Hooks

Give your components a memory and superpowers.

Chapter 4: the most important modern React feature.

What are Hooks?

Special functions that let a component remember things and do things. Names always start with use.



useState

Remember a value



useEffect

Do something after showing



useRef

Grab an element



useContext

Share data everywhere



2 rules: (1) call hooks at the TOP of your component, never inside if or loops. (2) only inside React components.

useState: the memory of a component

```
const [count,
```

```
setCount] =  
useState(0);
```



count

The current value. Read it to show on
screen.



setCount

setCount

The function to change it. React then re-draws.



useState(0)

The starting value is 0.



Never write `count = 5`. Always use `setCount(5)`, or the screen won't update!

The square brackets are array destructuring: `useState` gives back two things.

Example: a counter

with useState

Counter.jsx

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count +
1)}>+1</button>
      <button onClick={() => setCount(count -
1)}>-1</button>
```

```
<button onClick={() => setCount(0)}>Reset</button>  
</div>  
);  
}
```

1

onClick runs

It calls `setCount(count + 1)`

2

State updates

React saves the new count

3

Re-render

The screen shows the new number

Your turn: click the counter!

It plays by itself. Click any button to take control 🖱️

Let a student come and click. Point at the state box changing before the screen.

useEffect: do something after the

screen shows

useEffect

```
useEffect(() => {  
  // code to run  
}, [dependencies]);
```

For side effects: fetching data, timers,
changing the page title.



Like saying: "After you finish drawing,
THEN do this."

Dependency list	When does it run?	Good for
[] empty	Only ONCE, when it first shows	Fetching data
[count]	Every time count changes	Updating the title
(no list)	After EVERY render	Rarely needed

Fetching a fake API:

part 1, get the data

Users.jsx

```
import { useState, useEffect } from "react";

function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((res) => res.json())
      .then((data) => {
        setUsers(data);
        setLoading(false);
      });
  });
}
```

```
});
```

```
}, []);
```



2 states

users starts empty, loading starts true.



useEffect + []

Runs once, when it appears.



fetch → **json** → **set**

Ask the server, read JSON, save it.

jsonplaceholder is a free fake API for practice. Same `fetch()` as our AJAX lesson, now inside `useEffect`.

Fetching a fake API: part 2, show the data

Users.jsx

```
// still inside Users()
```

```
if (loading) {
```

```
return <p>Loading...</p>;  
}  
return (  
  <ul>  
    {users.map((user) => (  
      <li key={user.id}>{user.name}</li>  
    ))}  
  </ul>  
);  
}
```



Loading first

While waiting, show a message.



map() makes a list

Each user becomes one `<li>`.



key = unique id

Helps React track each item.

Watch the data arrive

Component shows → `useEffect` calls

fetch() → server sends JSON →
setUsers() → list appears.

Two more hooks in one minute

 useRef: grab an element

Search.jsx

```
const inputRef = useRef(null);
```

```
<input ref={inputRef} />
```

```
<button onClick={() => inputRef.current.focus()}>
```

```
Focus
```

```
</button>
```

 useContext: share data everywhere

Theme.jsx

```
const ThemeContext = createContext("light");
```

```
<ThemeContext.Provider value="dark">
```

```
<App />
```

```
</ThemeContext.Provider>
```

```
// any child, any depth:
```

```
const theme = useContext(ThemeContext);
```



useContext solves "prop drilling": passing the same prop through many layers.

0

5

Chapter 5

Forms

Controlled inputs: React holds what the user types.

Chapter 5: user input the React way.

Controlled input: React holds the value

1. User types "R" a key is pressed
2. `onChange` runs `setName("R")`

3. State updates `name = "R"`

4. Input shows `itvalue={name}`

The state is the boss 

The input shows what state says

Example: a simple sign-up form

SignupForm.jsx

```
import { useState } from "react";  
function SignupForm() {  
  const [name, setName] = useState("");
```

```
function handleSubmit(e) {  
  e.preventDefault(); // stop page reload  
  alert("Hello, " + name + "!");  
}  
return (  
  <form onSubmit={handleSubmit}>  
    <input  
      value={name}  
      onChange={(e) => setName(e.target.value)}  
    />  
    <button type="submit">Send</button>  
  </form>  
  );  
}
```



value={name}

The input shows the state.



onChange

Runs on every key. `e.target.value` = what was typed.



e.preventDefault()

Stops the page reload. Same as jQuery!

Watch the form work

Each key press → onChange → setName
→ the input shows the new value.

Many inputs? Use one state object

Signup.jsx

```
const [form, setForm] = useState({ name: "", email: "" });  
function handleChange(e) {  
  setForm({ ...form, [e.target.name]: e.target.value });  
}  
  
<input name="name" value={form.name}
```

```
onChange={handleChange} />
```

```
<input name="email" value={form.email}
```

```
onChange={handleChange} />
```



One object

All fields live together in form.



name attribute

Tells us WHICH field changed.



...form

Copies old values, then changes one.

The spread operator `...form` copies all old fields. `[e.target.name]` picks the field by its name.

0

6

Chapter 6

Routing and more

Many pages in one app, plus lists,
keys and conditions.

Chapter 6: React Router and everyday patterns.

Routing: many
pages, no reloads

A router looks at the URL and decides which component to show.

React doesn't include one. We add the popular one:

Terminal

```
npm install react-router-dom
```



Like a receptionist who sends you to the right room.

URL: /

<Home />

URL: /about

<About />

URL: /contact

<Contact />

React Router in code

App.jsx

```
import { BrowserRouter, Routes, Route, Link }  
from "react-router-dom";  
function App() {  
  return (  
    <BrowserRouter>  
      <nav>  
        <Link to="/">Home</Link>  
        <Link to="/about">About</Link>  
      </nav>  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
      </Routes>  
    </BrowserRouter>  
  );  
}
```



BrowserRouter

Turns routing ON. Wrap the whole app.



Routes + Route

Which component for which URL.



Link

Like `<a>`, but with no page reload.

Watch React Router

switch pages

The URL changes, the matching Route is picked, and the reload counter stays at 0.

Showing lists: map() and key

FruitList.jsx

```
const fruits = ["Apple", "Mango", "Banana"];  
function FruitList() {
```

```
return (  
  <ul>  
    {fruits.map((fruit, index) => (  
      <li key={index}>{fruit}</li>  
    ))}  
  </ul>  
);  
}
```



map()

Turns each array item into JSX.



key

A unique label for each item. Use a real id when you have one.

Result on screen

 Apple

 Mango

 Banana

Show things only sometimes

Greeting.jsx

```
function Greeting({ isLoggedIn }) {  
  return (  
    <div>  
      {isLoggedIn ? <h1>Welcome back!</h1> :  
      <h1>Please log in</h1>}  
      {isLoggedIn && <button>Logout</button>}  
    </div>  
  );  
}
```

?

cond ? A : B

If true show A, else B. Like if/else.



cond && A

Show A only when true.



{ isLoggedIn }

Short way to write props.isLoggedIn.

6 common beginner mistakes

 Mistake

 Fix

Writing
`class="box"`

Use
`className="box"`

`count = count + 1`

`setCount(count + 1)`

Forgetting key in
a list

`<li key={item.id}>`

`useEffect` with
`fetch` but no `[]`

Add `[]` so it runs
once, not forever

Returning two

Wrap them in `<>`

 Mistake	 Fix
tags side by side	... </>
Using <a href> for pages	Use <Link to> from React Router

These cause most beginner bugs. Keep this slide handy while practising.

Cheat sheet: React on one page

Concept	Meaning	Example
Component	A function that returns JSX	<code>function Card() {...}</code>
JSX	HTML-like code inside JS	<code><h1>{title}</h1></code>
Props	Data sent into a component	<code><Card name="Ravi" /></code>
State	A value that can change	<code>useState(0)</code>
useEffect	Run code after render	<code>useEffect(() => {...}, [])</code>

Concept	Meaning	Example
Event	React to clicks/typing	<code>onClick={handleClick}</code>
Controlled input	State holds the value	<code>value={name}</code> <code>onChange={...}</code>
List + key	Show an array	<code>items.map(i => <li key={i.id}>)</code>
Condition	Show only if true	<code>{ok && <p>Done</p>}</code>
Routing	URL picks a component	<code><Route path="/about" /></code>

Quick quiz: test yourself 🧠

Q1

Are React and React.js different?

click to flip



No! Same thing, two names.

Q2

A component name must start with...?

click to flip



A CAPITAL letter, like Welcome.

Q3

How do you send data into a component?

click to flip



With props: name="Ravi"

Q4

Why `setCount` and not `count = 5`?

click to flip



`setCount` tells React to re-render.

Q5

When does `useEffect(..., [])` run?

click to flip



Only once, when it first shows.

Q6

What does `e.preventDefault()` do?

click to flip



Stops the page from reloading.

Q7

Why Link instead of `<a>`?

click to flip



Link changes pages with no reload.

Q8

What is diffing?

click to flip



Comparing old vs new Virtual DOM.

Think first, then click a card to see the answer. Flip all 8 for a surprise! 🎁

Students answer aloud, then click to flip. Press A to flip all. All 8 flipped = confetti!

Mini project: build a To-Do app



Use everything you learned, one step at a time:

1

Create the app with Vite

Uses: Setup

2

Make a TodoApp component

Uses: Components

3

Store todos in useState([])

Uses: State

4

Input + Add button

Uses: Forms, events

5

Show todos with map() + key

Uses: Lists

6

Bonus: load todos from jsonplaceholder

Uses: useEffect, fetch

Extra challenge: a delete button for each todo, and an About page with React Router.



You did it!

You now know the React basics. Next:
build the To-Do app, practise a little every
day, and explore react.dev.

 Components  State  Effects  Forms  Routing
Congratulations! The best way to master React is to build small projects.